COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Sep 10, 2026	**DATE SUBMITTED:**	Sep 22, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8 **Sorting Algorithms Part 2**

1. PROCEDURE OUTPUT

Code

```
const int max_size = 100;
void display(int list[]){
    for(int i = 0; i < max_size; i++ ){
        std::cout << list[i] << " ";
        if((i+1) % 10 == 0){
            std::cout << std::endl;
        }
    }
}
```

```
int main(){
    int randArr[max_size];
    srand(time(0));

    for(int i = 0; i < max_size; i++){
        randArr[i] = rand()%10;
    }
    std::cout << "Array of Values for Sort Algorithm Testing: " << std::endl;
    display(randArr);
    std::cout << std:: endl;
```

Output

```
Array of Values for Sort Algorithm Testing:
3 2 2 6 3 2 4 7 8 2
3 9 5 4 9 2 3 4 3 5
5 9 5 8 5 8 0 1 8 0
0 1 7 9 6 4 1 1 7 4
3 1 6 7 9 4 5 4 3 3
0 7 0 8 3 7 9 8 0 5
6 9 5 4 9 8 1 4 7 1
3 4 2 3 1 6 8 6 5 7
9 9 3 4 1 0 1 4 1 2
0 5 5 2 7 5 7 1 3 1
```

Observations	This code makes a random array of 100 integers, each between 0-9, using rand() % 10, so it's testing the sort algorithms on a much bigger array than the 15-element one from before. The display function prints a line break every 10 values, so it's easier to read as a grid instead of one giant line. Since srand(time(0)) seeds the generator with the current time, the array changes every time you run the program, meaning it's not reproducible like the hardcoded one but tests the sorts on more random data. Because the values are limited to 0-9, there are a ton of duplicates in the array, and this can affect how a sort performs, especially quick sort, where a lot of equal elements can make the partitions less balanced compared to an array where every value is unique.

Table 8-1. Array of Values for Sort Algorithm Testing

Code

```
template <typename T>
void shellSort(T arr[], int arrSize){
    for(int gap = arrSize/2; gap > 0; gap /= 2){
        for(int i = 0; i < arrSize; i++){
            int temp = arr[i];
            int j = i;

            while(j >= gap && arr[j - gap] > temp){
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}
```

```
std::cout << "Shell Sort: " << std::endl;
shellSort(randArr, max_size);
display(randArr);
```

Output

```
Array of Values for Sort Algorithm Testing:
3 2 2 6 3 2 4 7 8 2
3 9 5 4 9 2 3 4 3 5
5 9 5 8 5 8 0 1 8 0
0 1 7 9 6 4 1 1 7 4
3 1 6 7 9 4 5 4 3 3
0 7 0 8 3 7 9 8 0 5
6 9 5 4 9 8 1 4 7 1
3 4 2 3 1 6 8 6 5 7
9 9 3 4 1 0 1 4 1 2
0 5 5 2 7 5 7 1 3 1
```

```
Shell Sort:
0 0 0 0 0 0 0 0 1 1
1 1 1 1 1 1 1 1 1 1
1 2 2 2 2 2 2 2 2 3
3 3 3 3 3 3 3 3 3 3
3 3 4 4 4 4 4 4 4 4
4 4 4 4 5 5 5 5 5 5
5 5 5 5 5 5 6 6 6 6
6 6 7 7 7 7 7 7 7 7
7 7 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9
```

Observations	This sorting technique is basically an upgraded version of insertion sort, but it uses a gap that allows it to cover more of the array and compare elements that are farther apart. As it passes through the whole array, the gap continues to shrink until it becomes 1, and then it becomes a regular insertion sort.

Table 8-2. Shell Sort Technique

Code

```
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int leftSize = mid - left + 1;
    int rightSize = right - mid;

    T leftArray[leftSize];
    T rightArray[rightSize];

    for(int i = 0; i < leftSize; i++){
        leftArray[i] = arr[left + i];
    }

    for(int i = 0; i < rightSize; i++){
        rightArray[i] = arr[mid + 1 + i];
    }

    int i = 0, j = 0, k = left;
    while(i < leftSize && j < rightSize){
        if(leftArray[i] <= rightArray[j]){
            arr[k] = leftArray[i];
            i++;
        }else{
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }

    while(i < leftSize){
        arr[k] = leftArray[i];
        i++;
        k++;
    }

    while(j < rightSize){
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}
```

```
template <typename T>
void mergeSort(T arr[], int left, int right){
    if(left < right){
        int middle = left + (right - left) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle+1, right);
        merge(arr, left, middle, right);
    }
}
```

```
std::cout << "Merge Sort: " << std::endl;
mergeSort(randArr, 0, max_size);
display(randArr);
```

Output

```
Array of Values for Sort Algorithm Testing:
3 4 0 3 5 8 9 2 8 3
8 0 8 6 5 3 4 5 9 1
9 4 3 2 6 1 4 8 0 7
6 8 3 8 9 6 2 3 2 4
9 7 7 1 2 0 5 0 9 9
3 4 2 1 3 8 3 3 1 8
5 5 7 1 2 9 0 0 5 4
5 7 0 4 0 1 2 9 2 9
4 5 4 1 0 5 8 7 5 5
9 6 8 0 4 9 7 3 6 7
```

```
Merge Sort:
0 0 0 0 0 0 0 0 0 0
0 1 1 1 1 1 1 1 1 2
2 2 2 2 2 2 2 2 3 3
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
4 5 5 5 5 5 5 5 5 5
5 5 5 6 6 6 6 6 6 7
7 7 7 7 7 7 7 8 8 8
8 8 8 8 8 8 8 8 9 9
9 9 9 9 9 9 9 9 9 9
```

Observations

This sorting technique uses the concept of divide and conquer, and it's also recursive, which means it calls itself over and over. It keeps halving the array down until it's just single elements, then sorts and merges everything back together on the way up.

Table 8-3. Merge Sort Algorithm

Code

```
template <typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}
```

```
template <typename T>
void quickSort(T arr[], int low, int high){
    if(low < high){
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

```
std::cout << "Quick Sort: " << std::endl;
quickSort(randArr, 0, max_size);
display(randArr);
```

Output

```
Array of Values for Sort Algorithm Testing:
0 5 3 5 1 5 2 2 5 5
0 3 2 1 0 5 9 1 3 1
3 6 8 1 2 0 0 5 1 6
5 5 3 9 0 6 7 9 7 3
1 2 7 1 2 5 2 2 0 2
4 9 0 4 8 5 7 1 5 4
7 6 3 0 0 3 5 9 8 9
3 0 5 2 9 3 8 6 5 5
0 8 5 7 5 0 4 8 2 7
8 9 5 6 4 9 0 9 2 2
```

```
Quick Sort:
0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 1 1 1 1 1
1 1 1 2 2 2 2 2 2 2
2 2 2 2 2 2 3 3 3 3
3 3 3 3 3 3 4 4 4 4
4 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 7 7 7 7
7 7 7 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9
```

Observations

This sorting technique uses a pivot to sort the array. It takes the last element as the pivot, then checks the elements before it and moves the values smaller than the pivot to the left side. After going through the array, it places the pivot in its correct position. It then repeats the same process on the parts before and after the pivot until the whole array is sorted.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer

```
std::cout << "Array of Values for Sort Algorithm Testing: " << std::endl;
display(randArr);
std::cout << std:: endl;

std::cout << "Quick Sort Modified with Merge Sort: " << std::endl;
quickSortModified(randArr, 0, max_size);
display(randArr);
```

```
template <typename T>
void quickSortModified(T arr[], int low, int high){
    if(low < high){
        int pivot = partition(arr, low, high);
        mergeSort(arr, low, pivot - 1);
        mergeSort(arr, pivot + 1, high);
    }
}
```

```
Array of Values for Sort Algorithm Testing:
0 5 0 2 0 6 0 7 1 1
2 3 4 8 5 2 7 3 8 4
0 1 6 4 0 7 3 9 3 0
0 9 9 8 9 6 2 1 7 4
8 4 2 1 5 2 7 3 6 7
4 2 8 7 2 5 8 5 8 2
4 9 7 8 2 1 2 9 5 1
4 1 5 3 5 2 8 5 0 9
4 9 4 2 8 5 1 5 6 7
4 2 5 5 6 4 8 4 7 6
```

```
Quick Sort Modified with Merge Sort:
0 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 2 2
2 2 2 2 2 2 2 2 2 2
2 2 3 3 3 3 3 3 4 4
4 4 4 4 4 4 4 4 4 4
4 5 5 5 5 5 5 5 5 5
5 5 5 5 6 6 6 6 6 6
6 7 7 7 7 7 7 7 7 7
7 8 8 8 8 8 8 8 8 8
8 8 9 9 9 9 9 9 9 9
```

We can sort the left and right sub list from the partition method in quick sort using, for example, merge sort. Using the pivot, we can identify the high or low variable needed in merge sort. This hints that we can mix and match different sorting algorithms that may help us to find the most optimal solution when it comes to sorting algorithms.

Problem 2

Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Answer

```
std::cout << "Given Array: " << std::endl;
int arrGiven[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
int shellArr[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
int mergeArr[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
int quickArr[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};

display(arrGiven, 15);
std::cout << std:: endl;

std::cout << "Shell Sort: "<< std::endl;
shellSort(shellArr, 15);
display(shellArr, 15);
std::cout << std:: endl;

std::cout << "Merge Sort: "<< std::endl;
mergeSort(mergeArr, 0, 14);
display(mergeArr, 15);
std::cout << std:: endl;

std::cout << "Quick Sort: "<< std::endl;
quickSort(quickArr, 0, 14);
display(quickArr, 15);
std::cout << std:: endl;
```

```
Given Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Shell Sort:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
Merge Sort:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
Quick Sort:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

Quick sort is fastest here since the array isn't sorted, so it avoids its worst case, and it sorts in place with better cache performance than merge sort. Both hit O(N log N) because splitting the array in half repeatedly gives log N levels, and touching every element at each level (merging or partitioning) gives N work per level, so N times log N is N log N. Merge sort always splits evenly, so it's guaranteed O(N log N); quick sort only gets that with a good pivot, since a bad one turns log N levels into N levels, causing O(N^2). You can also mix the two, sorting quick sort's sub-lists with merge sort using the pivot as the bounds.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Helped with explaining concepts that allowed me to understand and tackle the activity with better understanding..**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.